APEX v6.0 — Complete Manual

The Official Reference for APEX AI Engineering OS Updated: May 10, 2026 · Version 6.0.0

Table of Contents

---

1. What Is APEX

APEX is a framework that runs on top of Claude Code. It gives Claude Code what it fundamentally lacks:

- **Memory** — facts, decisions, patterns that persist across sessions
- **Enforcement** — rules that actually can't be broken (hooks, not suggestions)
- **Token awareness** — visibility and control over what you spend
- **Crash recovery** — you never lose your place again
- **Self-improvement** — the system learns from corrections automatically
- **Identity** — your AI is named whatever you want it to be

APEX does not replace how you use Claude Code. You still type commands in the terminal. Claude still writes your code. APEX is the infrastructure underneath that makes it reliable, consistent, and smart.

How APEX Works

When you install APEX into a project, it creates a `.claude/` directory containing:

- **Intelligence modules** (Python scripts that run locally — free, fast, no API calls)
- **Hook scripts** (shell scripts that enforce rules deterministically)
- **Brain files** (JSON Lines files that persist your project's knowledge)
- **Command files** (Markdown guides that Claude reads when you invoke a command)
- **Reference docs** (Technical docs injected only when relevant)

None of the Python modules make API calls. None of the hook scripts require network access. Everything local to your machine. The only API calls are the ones Claude Code makes naturally when you type commands.

---

2. Installation

Requirements

- Claude Code (any recent version, May 2026+)
- Python 3.8 or higher (`python3 --version`)
- Git (for crash recovery git hash tracking)
- Bash (macOS, Linux, or Windows Git Bash)

Fresh Install

```bash
```

```
# Step 1: Download APEX v6.0 from GitHub releases
# https://github.com/DevvNirvana/claude-code-apex/releases

# Step 2: Unzip (you get a folder: claude-orchestrator-apex-v6/)
unzip claude-orchestrator-apex-v6.0-COMPLETE.zip

# Step 3: Navigate to your project
cd /path/to/your-project

# Step 4: Run the installer
bash ~/claude-orchestrator-apex-v6/install.sh

# Step 5: Name your APEX
python3 .claude/intelligence/apex_identity.py setup

# Step 6: Generate enforcement hooks
python3 .claude/intelligence/hooks_generator.py

# Step 7: Open Claude Code and run first-time setup
/setup
```

The installer creates the complete `.claude/` directory structure. Your existing project files are never modified.

What Install Does

```
Creates:
  .claude/commands/          20 command guides
  .claude/intelligence/      19 Python modules
  .claude/references/        23 technical reference docs
  .claude/scripts/           Utility scripts
  .claude/hooks/             Empty (populated by hooks_generator.py)
  .claude/config/            Default config files
  .claude/skills/            Empty (populated by skills_manager.py)

Does NOT touch:
  Your source code
  Your existing CLAUDE.md (if any)
  Anything outside .claude/
```

install.sh Flags

```bash
# Full install (new project)
bash install.sh

# Update only (preserves brain, memory, identity)
bash install.sh --update

# Dry run (see what would happen without doing it)
bash install.sh --dry-run

# Install to specific directory
bash install.sh --dest /path/to/project/.claude
```

---

3. First Session

After installation, open Claude Code in your project and run:

```
/setup
```

This single command does everything:

1. Detects your tech stack (Next.js, React, Django, Rails, Go, FastAPI, etc.)
2. Generates a project-specific `CLAUDE.md` (~45 lines, not 200)
3. Seeds the brain with 8-12 stack-specific constraints
4. Warms the plan cache with common patterns for your stack
5. Sets up the identity if not already configured
6. Verifies hooks are active

After `/setup`, run `/init` at the start of every future session. This is your daily routine:

```
/init
```

`/init` does:

1. **Crash recovery check** — detects any crash from last session
2. **Identity banner** — shows your named AI (JARVIS, APEX, etc.)
3. **Token intelligence audit** — CLAUDE.md size, brain size, hooks status

4. **Brain sync** — validates facts are current
5. **Cache warm** — ensures semantic cache is ready

---

4. Identity & Personalization

APEX can be named anything. The identity system drives every visual element.

Setup Wizard

```bash
python3 .claude/intelligence/apex_identity.py setup
```

You'll be prompted for:

- **Name** — JARVIS, ALFRED, NOVA, FORGE, NEXUS, or anything
- **Tagline** — Short description ("Just A Rather Very Intelligent System")
- **Greeting** — What it says at session start ("Ready, boss.")
- **Owner name** — How it addresses you ("Dev", "Boss", your name)
- **Color scheme** — cyan, green, yellow, magenta, blue

Setting Individual Fields

```bash
python3 .claude/intelligence/apex_identity.py set name "JARVIS"
python3 .claude/intelligence/apex_identity.py set greeting "Online and ready."
python3 .claude/intelligence/apex_identity.py set color_scheme "cyan"
python3 .claude/intelligence/apex_identity.py set owner_name "Dev"
```

Viewing Current Identity

```bash
python3 .claude/intelligence/apex_identity.py show
python3 .claude/intelligence/apex_identity.py banner    # full startup banner
```

Per-Project Identities

Each project has its own `.claude/identity.json`. Project A can be JARVIS, Project B can be ALFRED. The name is per-project.

- Session start hook banner
- Token pre-flight header
- CLAUDE.md identity line (injected at top)
- All hook output prefixes
- Status command header

---

5. Token Intelligence & Smart Routing

This is the core system that prevents token waste.

The Smart Router

Before any context loads, the Smart Router classifies your task:

Tier	Tokens	USD	Loads
MICRO	~300	$0.001	Claude system prompt only
LIGHT	~900	$0.003	+ CLAUDE.md + minimal brain
STANDARD	~2,800	$0.008	+ full brain + skill body
FULL	~5,500	$0.017	Everything

MICRO tier examples (lookup, typo, simple question):

- "where is the auth code?"
- "what does useSession return?"
- "fix this spelling mistake"
- "which file handles routing?"

FULL tier examples (planning, design, review):

- `/plan build the authentication system`
- `/design the user dashboard`
- `/review before merging`
- `/execute the task list`

The router detects action verbs and bumps tier automatically. "implement a login page" typed as `/ask` still gets STANDARD treatment — it won't be misclassified as MICRO.

Pre-Flight Report

Before every command, see what you're about to spend:

```bash
python3 .claude/intelligence/token_intelligence.py pre-flight plan "build auth syste
```

Shows:

- Tier classification (MICRO/LIGHT/STANDARD/FULL)
- Exact token breakdown per component
- Dollar cost estimate
- Today's spend vs budget
- Any waste flags

Session Audit

```bash
python3 .claude/intelligence/token_intelligence.py audit
```

Shows:

- Today's spend and command count
- This week's total
- Breakdown by command type
- CLAUDE.md health (flag if over 600 tokens)
- Brain size (flag if over 500 tokens)
- Skills installed count
- Hooks active count

Budget Configuration

Edit `.claude/config/cache-config.json`:

```json
```

```
{
  "session_budget": {
    "soft_warn_usd": 3.00,
    "hard_halt_usd": 5.00
  }
}
```

`soft_warn_usd` — shows a warning but proceeds. `hard_halt_usd` — blocks the command entirely. Not a single token is spent.

When blocked, APEX shows:

```
⛔ BLOCKED: Hard limit: today $4.87 + this ~$0.17 = $5.04 ≥ $5.00
   Adjust limit in .claude/config/cache-config.json
```

CLAUDE.md Optimization

Keep CLAUDE.md under 50 lines. The optimizer detects and removes:

- Architecture folder trees (proven to hurt performance — ETH Zurich)
- Generic advice ("write clean code" — wastes instruction budget)
- Rules already enforced by hooks (redundant after hooks setup)

bash

```
# Check what can be improved
python3 .claude/intelligence/claude_md_optimizer.py --audit

# Apply optimizations (backs up original as CLAUDE.md.bak)
python3 .claude/intelligence/claude_md_optimizer.py --optimize
```

---

6. The Brain System

The brain is APEX's persistent memory. Facts survive sessions, get stronger with use, and get weaker when they cause problems.

Writing Facts

Brain facts are written via Claude Code commands. When you run `/setup`, APEX writes initial facts. As you work, `/plan`, `/review`, and `/debug` update the brain.

You can also write manually:

```bash
python3 .claude/intelligence/project_brain.py write \
  "All database queries through lib/db/queries.ts" \
  "constraint" \
  0.95
```

Arguments: content, category (constraint/pattern/decision/context), confidence (0.0-1.0)

Fact Categories

Category	What It Is	Example
`constraint`	Hard rules — things Claude must always/never do	"Never use API routes, use Supabase client directly"
`pattern`	Preferred approaches for this project	"All modals use shadcn Sheet component"
`decision`	Architecture decisions with reasoning	"Chose Supabase over Prisma for real-time support"
`context`	Background knowledge about the project	"Auth system uses NextAuth with Supabase adapter"

Reading Brain Context

```bash
# Full status
python3 .claude/intelligence/project_brain.py status

# Search for relevant facts
python3 .claude/intelligence/project_brain.py read "authentication"

# Check for conflicts
python3 .claude/intelligence/project_brain.py conflicts
```

Brain Delta Updates (ACE-Inspired)

Every fact has a confidence score that auto-adjusts based on usage:

```bash
```

```
# Mark a fact as having helped (confidence increases +0.02)
python3 .claude/intelligence/project_brain.py reinforce <fact-id> helpful

# Mark as harmful (confidence decreases -0.05)
python3 .claude/intelligence/project_brain.py reinforce <fact-id> harmful
```

After 3 harmful marks, a fact is flagged for review. Claude Code does this automatically when you correct its behavior.

Stale Facts

bash

```
# Find facts not reinforced in 30+ days
python3 .claude/intelligence/project_brain.py stale
```

Facts that haven't been used or reinforced in 30 days are candidates for pruning. Old constraints that no longer apply waste instruction budget.

---

7. The Hooks System

Hooks are the key difference between APEX and a simple prompt collection. CLAUDE.md rules are suggestions. Hooks are laws.

Research finding (ETH Zurich, 2026):

- CLAUDE.md compliance: ~60%, degrades after 5 messages, vanishes after compaction
- Hook compliance: ~90%+, session-length independent, survives compaction

Generating Hooks

bash

```
# Generate all hooks from brain constraints
python3 .claude/intelligence/hooks_generator.py

# Preview without writing
python3 .claude/intelligence/hooks_generator.py --preview

# Overwrite existing hooks
python3 .claude/intelligence/hooks_generator.py --force

# Check hook status
python3 .claude/intelligence/hooks_generator.py --audit
```

What Gets Generated

After running hooks_generator, you get `.claude/settings.json` and `.claude/hooks/`:

Hook Script	Trigger	What It Does
`session-start.sh`	SessionStart	Re-injects brain constraints. Runs update check.
`crash-checkpoint.sh`	PreToolUse (Write/Edit/Bash)	Writes atomic checkpoint before every edit
`check-secrets.sh`	PreToolUse (Write/Edit)	Blocks hardcoded API keys, tokens
`protect-main.sh`	PreToolUse (Bash)	Blocks `git push origin main`
`inject-reference.sh`	UserPromptSubmit	Injects relevant reference doc per message
`session-pollution.sh`	UserPromptSubmit	Warns at turn 15, 25
`session-end.sh`	Stop	Docs reminder + crash guard clear

How Reference Injection Works

The `inject-reference.sh` hook fires before every message. It scans your words for keywords and injects exactly one relevant doc:

Keywords in your message	Doc injected
supabase, postgresql, sql, query	`sql-patterns.md`
react, component, hook, useState	`react-guidelines.md`
nextjs, app router, server component	`nextjs-guidelines.md`
test, jest, pytest, vitest	`testing-patterns.md`
tailwind, shadcn, css	`shadcn-tailwind-guidelines.md`
security, auth, rls, secret	`security-checklist.md`
(no match)	Nothing — zero tokens

Without this: 23 docs × ~1,300 tokens average = 30,000 tokens if all loaded.
With this: 1 doc × ~1,800 tokens = 1,800 tokens. ~28,000 tokens saved.

Removing Hook-Redundant Rules from CLAUDE.md

After generating hooks, remove these from CLAUDE.md — they're now enforced, not hoped for:

- "Never commit .env or API keys" → covered by `check-secrets.sh`
- "Never push directly to main" → covered by `protect-main.sh`

Removing them saves tokens AND improves compliance for your remaining rules.

---

8. Crash Recovery

How APEX Handles Crashes

Claude Code's native `--resume` is unreliable after OOM kills because `sessions-index.json` can become corrupted independently of the actual session files. APEX maintains its own crash state independently.

What happens during a session:

1. You run `/execute` to implement a feature
2. Your machine runs out of memory mid-task
3. Claude Code crashes
4. `sessions-index.json` may be corrupted → `claude --resume` fails

With APEX crash recovery:

1. Before every file edit, `crash-checkpoint.sh` fires (PreToolUse hook)
2. `crash_guard.py checkpoint` writes `.claude/checkpoints/last.json` atomically
3. Uses `os.replace()` — survives OOM kill mid-write
4. On restart, `/init` calls `crash_guard.py detect`
5. If crash detected, you see exactly where you were

What You See After a Crash

```
── APEX Crash Recovery Detected ──────────────────────
  Previous session ended unexpectedly.

  Last known state:
    Time:     2026-05-10 14:32:11
    Command:  /execute
    Task:     Step 3/5: Add RLS policies to profiles table
    Branch:   feat/auth
    Git hash: a3b4c5d

  Files that were being modified:
    • src/auth/session.ts
    • lib/db/policies.sql

  In-progress tasks at crash time:
    - [>] TASK-003: Add RLS policies

  To resume the Claude Code session:
  claude --resume abc123def456
  (may not work if session was corrupted)

  Recommended next steps:
  1. git diff HEAD to verify file state
  2. Review TODO.md for in-progress tasks
  3. Run /execute to continue
  4. python3 .claude/intelligence/crash_guard.py clear when done
──────────────────────────────────────────────────────
```

Manual Checkpoint Operations

bash

```
# Write a checkpoint manually (before a risky operation)
python3 .claude/intelligence/crash_guard.py checkpoint "execute" "About to drop inde

# Detect crash state
python3 .claude/intelligence/crash_guard.py detect

# Clear after clean completion
python3 .claude/intelligence/crash_guard.py clear

# View checkpoint history (last 10)
python3 .claude/intelligence/crash_guard.py history
```

Checkpoint Ring Buffer

APEX keeps the last 10 checkpoints in `.claude/checkpoints/archive/`. If `last.json` gets corrupted somehow, you have 9 previous states to fall back on.

---

9. All 20 Commands

/setup

When: First time in a project, or after major stack changes. **What it does:** Detects stack, generates CLAUDE.md, seeds brain, installs skills, warms cache. **Token tier:** FULL

```
/setup
```

/init

When: Start of every session. **What it does:** Crash detection → identity banner → token audit → brain sync → hook verification. **Token tier:** LIGHT

```
/init
```

/status

When: Anytime you want a system overview or proactive anomaly detection. **What it does:** Brain health, cache stats, quality grades, DORA metrics, stale tasks, budget trajectory, hook status. Surfaces anomalies without being asked. **Token tier:** LIGHT

```
/status
```

/ask

When: Read-only questions, code lookups, explanations. **What it does:** Answers from codebase context without writing anything. **Token tier:** MICRO (simple questions) or LIGHT (complex queries)

```
/ask "where does the session middleware live?"
/ask "what does the useAuth hook return?"
```

/brainstorm

When: Before planning any non-trivial feature. **What it does:** Socratic requirements gathering. Generates a Decision Record: what we ARE and ARE NOT building, why, and what traps to avoid. **Token tier:** FULL

```
/brainstorm "real-time presence indicators for the dashboard"
```

/plan

When: After brainstorm, or for any task with 2+ steps. **What it does:** DAG-structured task list with dependencies, estimated complexity, file paths. Reads brain constraints and past trajectories before planning. **Token tier:** FULL

```
/plan "implement user presence with Supabase realtime"
```

/execute

When: Running a plan from TODO.md or AI_TASKS.md. **What it does:** Batched task execution with lint/test checkpoints between steps. Detects context boundaries (unrelated consecutive tasks) and suggests fresh session. **Token tier:** FULL

```
/execute
```

/design

When: Building any UI — components, pages, layouts. **What it does:** Aesthetic direction phase before coding (picks visual direction). Then implements with your stack's design system tokens. **Token tier:** FULL

```
/design "user profile settings page"
```

/spawn

When: Multiple independent tasks that can run simultaneously. **What it does:** Creates parallel Claude agents in isolated git worktrees. Domain conflict detection prevents race conditions. **Token tier:** FULL

```
/spawn "TASK-001, TASK-002, TASK-003 can run in parallel"
```

/test

When: Writing tests for new or existing code. **What it does:** Framework-specific test generation (Jest, Vitest, Pytest, RSpec, etc.). TDD enforcement — tests before implementation. **Token tier:** STANDARD

```
/test "unit tests for the auth middleware"
```

/debug

When: Investigating a bug, error, or unexpected behavior. **What it does:** Root cause analysis with brain constraint injection. Checks if the bug violates a known constraint before looking elsewhere. **Token tier:** STANDARD

```
/debug "TypeError: Cannot read properties of undefined at session.ts:42"
```

/optimize

When: Performance issues — load time, query speed, bundle size. **What it does:** Profiling-guided optimization. Measures before, proposes targeted fixes, measures after. **Token tier:** STANDARD

```
/optimize "the profile query is slow on users with many connections"
```

/refactor

When: Improving code structure without changing behavior. **What it does:** Impact analysis, dependency checking, safe transformation. Verifies tests still pass. **Token tier:** STANDARD

```
/refactor "extract the auth logic from the API routes into a service layer"
```

/docs

When: Generating or updating documentation. **What it does:** README, API docs, JSDoc, inline comments. Style matches your existing docs. **Token tier:** STANDARD

```
/docs "generate JSDoc for lib/auth/session.ts"
```

/review

When: Before merging any significant change. **What it does:** Multi-perspective review: security, correctness, conventions (your AI_RULES.md), performance, simplicity. Grades A-F. **Token tier:** FULL

```
/review "the auth implementation before we merge feat/auth"
```

/ship

When: Before every deployment. **What it does:** 40-point pre-flight checklist. Runs your actual build and test commands. SHIP/HOLD/HOLD_CRITICAL verdict. **Token tier:** FULL

```
/ship
```

/rollback

When: A deploy broke production. **What it does:** Emergency rollback using worktree metadata and git revert. Preserves the broken state for analysis. **Token tier:** LIGHT

```
/rollback "the v2.3.1 deploy broke the auth flow"
```

/compact

When: TODO.md or SESSION_LOG.md exceeds 150 lines. **What it does:** Archives completed work, compresses stale docs, keeps active context clean. **Token tier:** LIGHT

```
/compact
```

/handoff

When: End of a complex session, before starting fresh. **What it does:** Creates `.claude/handoff.md` (under 400 tokens) with what shipped, what's active, what failed, what to do next. `/init` reads it automatically. **Token tier:** LIGHT

```
/handoff
```

/optimize-context

When: Costs are creeping up or compliance is dropping. **What it does:** Full audit of CLAUDE.md

(architecture trees, generic advice, redundant rules) + hooks status + token efficiency report.
Token tier: LIGHT

```
/optimize-context
```

---

10. All 19 Intelligence Modules

All modules live in `.claude/intelligence/`. All are local Python — no API calls. All have CLI interfaces.

smart_router.py

Token tier classifier. Runs before every command.

bash

```
python3 .claude/intelligence/smart_router.py ask "where is auth?"
python3 .claude/intelligence/smart_router.py plan "build auth system"
```

crash_guard.py

Atomic checkpoint writer and crash detector.

bash

```
python3 .claude/intelligence/crash_guard.py detect
python3 .claude/intelligence/crash_guard.py history
python3 .claude/intelligence/crash_guard.py clear
```

apex_identity.py

Custom naming and personalization engine.

bash

```
python3 .claude/intelligence/apex_identity.py setup
python3 .claude/intelligence/apex_identity.py show
python3 .claude/intelligence/apex_identity.py set name JARVIS
python3 .claude/intelligence/apex_identity.py banner
```

update_checker.py

Auto-update notifications with 24h cache.

```bash
python3 .claude/intelligence/update_checker.py check
python3 .claude/intelligence/update_checker.py version
python3 .claude/intelligence/update_checker.py set 6.0.0
```

token_intelligence.py

Pre-flight token reports and budget enforcement.

```bash
python3 .claude/intelligence/token_intelligence.py audit
python3 .claude/intelligence/token_intelligence.py pre-flight plan "build auth"
python3 .claude/intelligence/token_intelligence.py report execute
```

skills_manager.py

Converts commands to lazy-loaded Skills format.

```bash
python3 .claude/intelligence/skills_manager.py install
python3 .claude/intelligence/skills_manager.py stats
python3 .claude/intelligence/skills_manager.py validate
```

hooks_generator.py

Generates settings.json and hook scripts from brain constraints.

```bash
python3 .claude/intelligence/hooks_generator.py
python3 .claude/intelligence/hooks_generator.py --force
python3 .claude/intelligence/hooks_generator.py --preview
python3 .claude/intelligence/hooks_generator.py --audit
```

claude_md_optimizer.py

Research-backed CLAUDE.md trimmer.

```bash
python3 .claude/intelligence/claude_md_optimizer.py --audit
python3 .claude/intelligence/claude_md_optimizer.py --optimize
```

generate_claude_md.py

Auto-generates CLAUDE.md from stack detection.

```bash
python3 .claude/intelligence/generate_claude_md.py
python3 .claude/intelligence/generate_claude_md.py --name JARVIS
python3 .claude/intelligence/generate_claude_md.py --dry-run
```

project_brain.py

Persistent fact store with confidence delta updates.

```bash
python3 .claude/intelligence/project_brain.py status
python3 .claude/intelligence/project_brain.py write "constraint text" constraint 0.9
python3 .claude/intelligence/project_brain.py read "query terms"
python3 .claude/intelligence/project_brain.py conflicts
python3 .claude/intelligence/project_brain.py stale
```

trajectory_store.py

Experience replay and ACE Reflector.

```bash
python3 .claude/intelligence/trajectory_store.py stats
python3 .claude/intelligence/trajectory_store.py store <file>
# ACE Reflector (auto-called by /ship HOLD verdict):
python3 .claude/intelligence/trajectory_store.py reflect "task" "what failed" "what
```

cache_manager.py

Semantic plan cache. Avoids re-planning identical tasks.

```bash
python3 .claude/intelligence/cache_manager.py stats
python3 .claude/intelligence/cache_manager.py check "implement auth" plan
python3 .claude/intelligence/cache_manager.py clear
```

Session cost tracking and DORA metrics.

```bash
python3 .claude/intelligence/token_tracker.py report
python3 .claude/intelligence/token_tracker.py today
python3 .claude/intelligence/token_tracker.py week
```

detect_stack.py

Detects 15+ frameworks and versions from your project files.

```bash
python3 .claude/intelligence/detect_stack.py
```

evaluator.py

Self-scoring quality engine. Grades command output A-F.

```bash
python3 .claude/intelligence/evaluator.py score "output text" plan
python3 .claude/intelligence/evaluator.py history
python3 .claude/intelligence/evaluator.py trend
```

taste_memory.py

Learns your preferences from correction patterns.

```bash
python3 .claude/intelligence/taste_memory.py profile
python3 .claude/intelligence/taste_memory.py signal "prefer functional components"
```

benchmark.py

Statistical consistency measurement for commands.

```bash
python3 .claude/intelligence/benchmark.py run plan 5
python3 .claude/intelligence/benchmark.py compare plan review
```

design_system.py

Extracts design tokens from your project's config.

```bash
python3 .claude/intelligence/design_system.py extract
python3 .claude/intelligence/design_system.py tokens
```

framework_lint.py

Framework-specific lint and convention rules.

```bash
python3 .claude/intelligence/framework_lint.py check
python3 .claude/intelligence/framework_lint.py rules nextjs
```

---

11. Session Management

Recommended Daily Flow

```
Start of day:
  /init                  → crash check + identity + audit

Planning new work:
  /brainstorm            → requirements first
  /plan                  → then plan
  /execute               → then execute

Mid-session checkpoints:
  /status                → proactive anomaly check
  /review                → before committing significant changes

End of session:
  /ship                  → before deploying
  /handoff               → if session was complex (creates briefing)
  (hooks write /compact reminder if docs exceed 150 lines)
```

Session Pollution

Sessions degrade as context accumulates. The session-pollution hook tracks turn count:

After warning, use `/handoff` to create a context briefing, then start a new session. The briefing is under 400 tokens and gives the next session full context without the conversation bloat.

Context Boundaries in /execute

If `/execute` runs task A then task B and they're semantically unrelated (similarity score < 0.2), APEX surfaces:

```
⚠ APEX: Low similarity between tasks
  Previous: TASK-001 (auth implementation)
  Current:  TASK-005 (email template design)

  Options:
  a) Continue in this session
  b) /handoff then start fresh (recommended)
```

/compact

Run when your working docs get unwieldy:

```
/compact
```

Archives: completed tasks from TODO.md, session log entries older than 2 weeks, stale brain facts. Keeps: active tasks, recent decisions, current constraints. Result: clean context for the next session.

---

12. Budget Configuration

Full configuration in `.claude/config/cache-config.json`:

```json
```

```
{
  "cache": {
    "similarity_threshold": 0.85,
    "max_entries": 500,
    "ttl_hours": 168
  },
  "session_budget": {
    "soft_warn_usd": 3.00,
    "hard_halt_usd": 5.00,
    "monthly_target_usd": 50.00
  },
  "smart_routing": {
    "enabled": true,
    "micro_max_tokens": 350,
    "light_max_tokens": 900,
    "standard_max_tokens": 2800
  },
  "brain": {
    "max_facts": 100,
    "stale_days": 30,
    "high_confidence_threshold": 0.95
  }
}
```

Typical Cost Ranges (May 2026, Sonnet 4.6 pricing)

Usage Pattern	Daily Cost	Monthly Est.
Light (5-10 simple queries)	$0.05-0.15	$1-5
Medium (20-30 mixed commands)	$0.30-0.80	$9-25
Heavy (50+ commands, complex work)	$1.00-3.00	$30-90
Very heavy (all-day agentic runs)	$3.00-5.00	$90-150

With Smart Router, expect 30-50% reduction vs vanilla Claude Code for mixed sessions.

13. Upgrading

From v5.x to v6.0

```bash
cd your-project

# Step 1: Backup
mkdir -p .apex-backup-$(date +%Y%m%d)
cp -r .claude/brain .claude/memory .apex-backup-$(date +%Y%m%d)/
cp CLAUDE.md .apex-backup-$(date +%Y%m%d)/
cp -r .claude/identity.json .apex-backup-$(date +%Y%m%d)/ 2>/dev/null || true

# Step 2: Update system files (preserves everything above)
bash ~/claude-orchestrator-apex-v6/install.sh --update

# Step 3: Regenerate hooks (new crash-checkpoint hook added)
python3 .claude/intelligence/hooks_generator.py --force

# Step 4: Install new skills
python3 .claude/intelligence/skills_manager.py install

# Step 5: Set version marker
python3 .claude/intelligence/update_checker.py set 6.0.0

# Step 6: Verify
python3 .claude/intelligence/token_intelligence.py audit
/init
```

From v4.x to v6.0

Same steps as v5→v6. All versions use the same `.claude/brain/facts.jsonl` and `.claude/memory/` format.

What Survives Every Upgrade

- `.claude/brain/facts.jsonl` — all your facts, constraints, patterns
- `.claude/memory/trajectories/` — all your stored sessions
- `.claude/memory/taste_profile.json` — your learned preferences
- `.claude/memory/evaluations.jsonl` — quality grade history
- `.claude/identity.json` — your custom identity
- `CLAUDE.md` — your project context file

- All your `docs/` directory

---

14. Auto-Updates

APEX checks for new releases once per 24 hours at session start. This happens via the `session-start.sh` hook.

What happens:

1. Session starts → hook fires → `update_checker.py check` runs
2. Checks 24h cache first (no network if recently checked)
3. If cached check says current → silent, nothing shown
4. If update available → shows prompt once

The update prompt:

```
── APEX Update Available ──────────────────────────
  Installed:  v6.0.0
  Available:  v6.1.0
  Releases:   https://github.com/DevvNirvana/claude-code-apex/releases
────────────────────────────────────────────────────

  Update now? [y/n/skip24h]
```

y — shows exact upgrade commands **n** — skips for this session, prompts again tomorrow **skip24h** — suppresses for 24 hours

Manual Check

```bash
python3 .claude/intelligence/update_checker.py check
python3 .claude/intelligence/update_checker.py check --force    # ignore 24h cache
python3 .claude/intelligence/update_checker.py version          # show installed ver
```

Network Behavior

- 3-second timeout — if GitHub is unreachable, silently skips
- Works entirely offline — 24h cache prevents hammering GitHub
- Never auto-installs — always requires explicit permission

15. Troubleshooting

"ModuleNotFoundError" when running intelligence scripts

```bash
# Make sure you're running from your project root
pwd                      # should be your project root
ls .claude/intelligence  # should list .py files

# Python version check
python3 --version        # needs 3.8+
```

Hooks not firing

```bash
# Check settings.json exists and is valid
cat .claude/settings.json | python3 -m json.tool

# Regenerate
python3 .claude/intelligence/hooks_generator.py --force

# Check Claude Code sees the hooks
# In Claude Code: /settings → should show hook configuration
```

Brain facts not persisting

```bash
# Check file exists and is writable
ls -la .claude/brain/facts.jsonl
python3 .claude/intelligence/project_brain.py status

# Verify JSON format is valid
python3 -c "
from pathlib import Path
for line in Path('.claude/brain/facts.jsonl').read_text().splitlines():
    import json; json.loads(line)
print('All facts valid JSON')
"
```

Token costs higher than expected

```bash
# Full audit
python3 .claude/intelligence/token_intelligence.py audit

# Check CLAUDE.md size
wc -l CLAUDE.md          # target: ≤50 lines
python3 .claude/intelligence/claude_md_optimizer.py --audit

# Check if skills are installed (lazy loading)
python3 .claude/intelligence/skills_manager.py stats

# Check smart routing is working
python3 .claude/intelligence/smart_router.py ask "simple question"
# Should show MICRO tier
```

Crash recovery false positive

```bash
# If /init shows crash recovery but you didn't crash
python3 .claude/intelligence/crash_guard.py clear

# Check what the checkpoint says
cat .claude/checkpoints/last.json
```

"--update flag preserved my brain but hooks are outdated"

```bash
# After --update, always regenerate hooks
python3 .claude/intelligence/hooks_generator.py --force
```

"inject-reference.sh injects nothing for my framework"

The hook pattern matches common terms. To add your framework:

1. Edit `.claude/hooks/inject-reference.sh`
2. Add a new `echo "$PROMPT_LOWER" | grep -qiE 'your|keywords' && inject "your-reference.md"` line
3. Add your reference doc to `.claude/references/`

16. Architecture Reference

Directory Structure (complete)

```
your-project/
├── CLAUDE.md                          Project context (40-50 lines, optimized)
└── .claude/
    ├── settings.json                  Claude Code hooks config (auto-generated)
    ├── identity.json                  Your APEX name and personality
    ├── commands/                      20 lazy-loaded command guides
    │   ├── init.md
    │   ├── plan.md
    │   └── ... (18 more)
    ├── intelligence/                  19 Python modules (the engine)
    │   ├── smart_router.py            Token tier classification
    │   ├── crash_guard.py             Crash recovery
    │   ├── apex_identity.py           Personalization
    │   ├── update_checker.py          Auto-updates
    │   ├── token_intelligence.py      Token reports + budget
    │   ├── skills_manager.py          Lazy loading
    │   ├── hooks_generator.py         Hook generation
    │   ├── claude_md_optimizer.py     CLAUDE.md trimmer
    │   ├── generate_claude_md.py      CLAUDE.md generator
    │   ├── project_brain.py           Persistent facts
    │   ├── trajectory_store.py        Session learning
    │   ├── cache_manager.py           Semantic cache
    │   ├── token_tracker.py           Cost tracking
    │   ├── detect_stack.py            Stack detection
    │   ├── evaluator.py               Quality scoring
    │   ├── taste_memory.py            Preference learning
    │   ├── benchmark.py               Consistency measurement
    │   ├── design_system.py           Design tokens
    │   └── framework_lint.py          Lint rules
    ├── hooks/                         Active enforcement scripts
    │   ├── session-start.sh
    │   ├── crash-checkpoint.sh
    │   ├── inject-reference.sh
    │   ├── session-pollution.sh
    │   ├── check-secrets.sh
    │   ├── protect-main.sh
    │   └── session-end.sh
    ├── brain/
```

```
    │   └── facts.jsonl              Persistent project knowledge
    ├── memory/
    │   ├── trajectories/           Past session records
    │   ├── taste_profile.json      Learned preferences
    │   ├── taste_signals.jsonl     Raw signal stream
    │   └── evaluations.jsonl       Quality grade history
    ├── checkpoints/
    │   ├── last.json               Latest checkpoint (atomic)
    │   ├── completed.flag          Clean session marker
    │   └── archive/                Ring buffer (last 10)
    ├── skills/                     Lazy-loaded command bodies
    │   ├── plan/SKILL.md
    │   └── ... (19 more)
    ├── references/                 23 technical reference docs
    │   ├── react-guidelines.md
    │   ├── sql-patterns.md
    │   └── ... (21 more)
    ├── config/
    │   ├── cache-config.json       Budget + cache settings
    │   ├── apex-version.json       Installed version marker
    │   ├── context-map.json        Context routing config
    │   ├── output-contracts.json   Expected output formats
    │   └── stack-profile.json      Detected stack (auto-generated)
    ├── cache/
    │   ├── token_log.json          Session cost history
    │   ├── update-check.json       Update check cache
    │   └── plan_cache.json         Semantic plan cache
    └── logs/
        └── session_turns.txt       Turn counter for pollution detection
```

Data Flow

```
User types command
       ↓
UserPromptSubmit hooks fire (inject-reference, session-pollution)
       ↓
Smart Router classifies tier (MICRO/LIGHT/STANDARD/FULL)
       ↓
Token pre-flight: budget check
       ↓ (if allowed)
Command skill body loads (lazy, from .claude/skills/)
       ↓
Brain context loads (selective, based on tier)
```

```
       ↓
Claude executes the command
       ↓
PreToolUse hooks fire before each file edit (crash-checkpoint, check-secrets,
protect-main)
       ↓
PostToolUse hooks fire after each edit (lint, if configured)
       ↓
Response generated
       ↓
Stop hook fires (session-end: docs reminder + crash_guard.clear)
```

Fact Confidence Lifecycle

```
brain_write() → confidence: 0.80 (default)
       ↓
brain_reinforce("helpful") × 3 → confidence: 0.86
       ↓
brain_reinforce("helpful") × 10 → confidence: 1.00 (capped)

brain_reinforce("harmful") × 1 → confidence: 0.75
       ↓
brain_reinforce("harmful") × 3 → _needs_review: True
       ↓
brain_reinforce("harmful") × 5 → effectively dead (confidence: 0.50)
```

APEX v6.0.0 — May 10, 2026 Built for engineers who ship.